

Assignment 1 – GAN Playground

Deep Neural Networks for Computer Graphics

Roi Moshe
Nadav Abayov

June 4, 2026

Part 1 – Deep Convolutional GAN

Task 1.1 – Data Augmentation

The `vanilla` branch of `get_data_loader` composes the following pipeline (in order): resize to a slightly larger canvas, take a random square crop back to `opts.image_size`, randomly flip horizontally, convert to a tensor in $[0, 1]$, then normalise to $[-1, 1]$ so the output range matches the generator's `tanh`.

```
1 load_size = int(1.1 * opts.image_size)
2 osize = [load_size, load_size]
3 train_transform = transforms.Compose([
4     transforms.Resize(osize, Image.BICUBIC),
5     transforms.RandomCrop(opts.image_size),
6     transforms.RandomHorizontalFlip(),
7     transforms.ToTensor(),
8     transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),
9 ])
```

Task 1.2a – Padding Derivation

For a 2-D convolution with input spatial size H_{in} , kernel size K , stride S , and padding P , the output size is

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2P - K}{S} \right\rfloor + 1. \quad (1)$$

The discriminator's first four convolutions must downsample by exactly a factor of two, i.e. $H_{\text{out}} = H_{\text{in}}/2$, with the prescribed $K = 4$ and $S = 2$. Substituting into (1) (and noting that H_{in} is even so the floor is exact):

$$\frac{H_{\text{in}}}{2} = \frac{H_{\text{in}} + 2P - 4}{2} + 1$$

$$H_{\text{in}} = H_{\text{in}} + 2P - 4 + 2$$

$$0 = 2P - 2$$

$$\boxed{P = 1.}$$

Sanity check on layer 1: $H_{\text{in}} = 64$ gives $\lfloor (64 + 2 - 4)/2 \rfloor + 1 = 31 + 1 = 32$, halving the spatial extent as required. The same holds for the remaining three downsampling layers.

Task 1.2b – DCDiscriminator

Using $P = 1$ for layers 1–4 and the helper `conv` from `models.py`, the discriminator follows the channel/spatial schedule in Table ???. The final `conv5` collapses the 4×4 feature map to a single scalar with $K = 4, S = 1, P = 0$ (and no normalisation or activation, since the loss is least-squares on the raw logit).

Layer	$C_{\text{in}} \rightarrow C_{\text{out}}$	K	S	P	Norm	Activation
conv1	$3 \rightarrow 32$	4	2	1	InstanceNorm	ReLU
conv2	$32 \rightarrow 64$	4	2	1	InstanceNorm	ReLU
conv3	$64 \rightarrow 128$	4	2	1	InstanceNorm	ReLU
conv4	$128 \rightarrow 256$	4	2	1	InstanceNorm	ReLU
conv5	$256 \rightarrow 1$	4	1	0	–	–

Table 1: DCDiscriminator layer schedule. Output spatial sizes follow the doubling-down sequence $64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 1$.

Task 1.3 – DCGenerator

The generator is the discriminator’s mirror image: it takes a noise vector $z \in \mathbb{R}^{\text{noise_size}}$ shaped as `(noise_size, 1, 1)` and progressively upsamples it to a $3 \times 64 \times 64$ image. Layers 2–5 follow the canonical “upsample then convolve” recipe: nearest-neighbour upsampling with `scale_factor=2` doubles the spatial extent, and a $K = 3, S = 1, P = 1$ convolution preserves it (verifying with (??): $\lfloor (2H_{\text{in}} + 2 - 3)/1 \rfloor + 1 = 2H_{\text{in}}$). This factored design avoids the well-known checkerboard artifacts that plain transposed convolutions can produce.

First layer: $1 \times 1 \rightarrow 4 \times 4$ with no upsample. Following the assignment hint, `up_conv1` is built with the `conv()` helper (i.e. a plain `Conv2d` with InstanceNorm and ReLU, *no Upsample* stage at all) and lets the convolution itself expand the spatial extent. From (??) with $H_{\text{in}} = 1, H_{\text{out}} = 4, S = 1$:

$$4 = (1 + 2P - K) + 1$$

$$\Rightarrow 2P - K = 2.$$

Choosing $K = 4$ gives $\boxed{P = 3}$. Sanity check: padding the 1×1 input on every side with three zeros yields a 7×7 tensor; sliding a 4×4 kernel with stride 1 produces $\lfloor (7 - 4)/1 \rfloor + 1 = 4$ output positions, as required.

Layer	$C_{\text{in}} \rightarrow C_{\text{out}}$	K	S	P	Up.	Norm	Act.
up_conv1	noise_size $\rightarrow$ 256	4	1	3	–	InstanceNorm	ReLU
up_conv2	$256 \rightarrow 128$	3	1	1	$\times 2$	InstanceNorm	ReLU
up_conv3	$128 \rightarrow 64$	3	1	1	$\times 2$	InstanceNorm	ReLU
up_conv4	$64 \rightarrow 32$	3	1	1	$\times 2$	InstanceNorm	ReLU
up_conv5	$32 \rightarrow 3$	3	1	1	$\times 2$	–	Tanh

Table 2: DCGenerator layer schedule. `up_conv1` is built with the `conv()` helper (no `Upsample` stage); the rest use `up_conv()` with the default `scale_factor=2`. Spatial sizes follow $1 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow 32 \rightarrow 64$. The final `Tanh` maps the output into $[-1, 1]$, matching the `Normalize((0.5, ...), (0.5, ...))` step in Task 1.1.

Task 1.4 – DCGAN Training Loop

The training loop alternates one discriminator step and one generator step per minibatch, both with the least-squares loss from Algorithm 1 of the assignment.

Discriminator update. Given m real images x and m generator samples $\tilde{x} = G(z)$ (with fresh noise $z \sim \mathcal{U}[-1, 1]^d$),

$$\begin{aligned} J_{\text{real}}^{(D)} &= \frac{1}{m} \sum_{i=1}^m (D(x^{(i)}) - 1)^2, \\ J_{\text{fake}}^{(D)} &= \frac{1}{m} \sum_{i=1}^m D(G(z^{(i)}))^2, \\ J^{(D)} &= \frac{1}{2} (J_{\text{real}}^{(D)} + J_{\text{fake}}^{(D)}). \end{aligned}$$

The fake batch is detached (`fake_images.detach()`) before the discriminator forward pass so the discriminator gradients do not leak into G 's parameters.

Generator update. A *fresh* noise batch z' is drawn, $\tilde{x}' = G(z')$ is generated, and

$$J^{(G)} = \frac{1}{m} \sum_{i=1}^m (D(G(z'^{(i)})) - 1)^2.$$

Here gradients must flow through both D and G , so no detach is used and only the generator parameters are stepped. Drawing new noise (rather than reusing the discriminator's batch) is what the pseudocode prescribes and avoids any subtle bias from over-fitting to the same samples in the same iteration.

Task 1.5 – Differentiable Augmentation

`prepare_images` is the hook the training loop calls on every batch of images before they reach the discriminator (both real and fake). With `--use_diffaug` it applies the DiffAugment policy 'color,translation,cutout'; otherwise it is a pass-through:

```
1 def prepare_images(images, opts):
2     if opts.use_diffaug:
3         return DiffAugment(images, policy=policy)
4     return images
```

Why this is different from Task 1.1. Standard data augmentation (Task 1.1) lives in the dataloader, so it only touches *real* images and is non-differentiable – no gradient ever flows through it. DiffAugment is applied *inside* the training loop to both real and fake images, and every transformation it uses (brightness/colour jitter, translation, cutout) is implemented with differentiable PyTorch ops. As a result, when the generator's loss is back-propagated through the discriminator, gradients continue to flow through the augmentation into G . This is what the original DiffAugment paper identifies as essential: if only D saw augmented images while G was trained against the un-augmented version, the two networks would be optimising different distributions and training would degrade.

Task 1.6 – W&B Logging

Logging is wired up at four sites in `vanilla_gan.py`:

1. **Run init** (top of `training_loop`, after the fixed-noise vector is created):

```
1 wandb.init(  
2     project='assignment1-dcgan',  
3     name=f'dcgan_diffaug={opts.use_diffaug}',  
4     config=vars(opts),  
5 )
```

The run name carries the boolean `opts.use_diffaug` so the with/without comparison is immediate in the dashboard, and `config=vars(opts)` serialises every command-line argument.

2. **Scalar losses** every `opts.log_step` iterations: `D/real_loss`, `D/fake_loss`, `D/total_loss`, `G/loss`, all keyed by `step=iteration`. The `D/` and `G/` prefixes group the four panels under “Discriminator” and “Generator” headers in W&B.
3. **Image grid** inside `save_samples` (every `opts.sample_every` iterations) using `wandb.Image(grid, caption=...)`. The grid is the same uint8 image that is also written to disk, so the dashboard sample matches the local artefact bit-for-bit.
4. **Run finish**: `wandb.finish()` after the training loop returns.

Both `wandb.log` calls pass `step=iteration`, so the loss-curve panel and the image panel share a common monotone time axis that lets the visual quality at iteration k be read off against the loss values at the same k .

Task 1.7 – Experiments & Report

The DCGAN was trained for 500 epochs both with and without `--use_diffaug` (otherwise identical hyperparameters: batch size 16, $\eta = 2 \times 10^{-4}$, Adam $(\beta_1, \beta_2) = (0.5, 0.999)$, `--data_preprocess vanilla`). All loss curves, image grids, and the discussion required by the assignment (loss-curve evolution, visual-quality progression, and the Task 1.1 vs Task 1.5 augmentation comparison) are collected in a public W&B Report:

W&B Report:

<https://api.wandb.ai/links/roimoshe1995-tel-aviv-university/0n0y7jgw>

The report visibility is set to *Anyone with the link can view*.

Loss-curve evolution. Both runs were trained for $\sim 6,500$ iterations ($500 \text{ epochs} \times \sim 13 \text{ iters/epoch}$ on the ~ 200 -image `grumpifyBprocessed` dataset, batch size 16). Under the least-squares loss, the LSGAN equilibrium $D \equiv 0.5$ corresponds to $D/\text{real} = D/\text{fake} = D/\text{total} = G = 0.25$, while a fully discriminator-dominated regime ($D(x) \rightarrow 1$, $D(G(z)) \rightarrow 0$) gives $D/\text{total} \rightarrow 0$ and $G \rightarrow 1$.

- The **baseline** run sits squarely in the discriminator-dominated regime: D/total settles around ~ 0.04 and even drifts slightly downward, D/real and D/fake both hover at ~ 0.03 – 0.05 , and G/loss saturates near ~ 1.0 . With only ~ 200 training images and no augmentation inside the training loop, the discriminator quickly memorises the dataset and the generator’s gradient signal collapses – the textbook small-data DCGAN failure mode the assignment warns about in Section 1.1.
- The **DiffAugment** run is markedly closer to equilibrium: $D/\text{total} \approx 0.15$, $G/\text{loss} \approx 0.65$, both stationary throughout training. The visible upward spikes in D/real and D/fake (occasional excursions to 0.3 – 0.5) are the expected consequence of randomised augmentations presenting batch-to-batch variation in difficulty – they are a feature, since they prevent the discriminator from locking onto memorised features. Crucially, D/total shows no downward trend.

Neither run reaches the ideal 0.25 equilibrium, which is consistent with the very small dataset and the absence of additional regularisers, but DiffAugment moves both networks substantially in the right direction.

Visual quality over training. Inspecting the `samples` grid at three checkpoints (early ~ 200 , mid ~ 2400 , late ~ 6400):

- **Iter ~ 200 .** Both runs produce coarse brown/black/white blobs with no recognisable structure. Indistinguishable from each other – training has not yet had time to differentiate the two regimes.
- **Iter ~ 2400 .** Both runs have learned the Grumpy-Cat face structure (eyes, ears, mouth in roughly the right places). The *baseline* grid already exhibits the classic small-data pathologies: purple/magenta chromatic noise around the eyes and ears and high uniformity across the 16 cells. The *DiffAugment* grid is softer, with more natural fur and background colours and visible cell-to-cell variation in pose and lighting.
- **Iter ~ 6400 .** Both runs sharpen further. The *baseline* samples now show pronounced cyan/magenta fringing along facial outlines and an “etched”, over-sharpened texture; the *DiffAugment* samples have cleaner colours, smoother fur, and more cell-to-cell variation in head tilt and shading.

Because the dataset is essentially photographs of a single cat, the modest diversity gap between the two runs cannot be read as full mode collapse; the more telling differences are the *artefact patterns*. The baseline’s chromatic fringing and over-sharpening are exactly what one would predict from the loss curves: with D winning decisively, G ’s gradient saturates, so G keeps pushing high-frequency texture without correcting any of the colour or shape errors D no longer cares about.

Standard dataset augmentation vs. DiffAugment. The two augmentation schemes used in this assignment differ along three axes:

- **Where applied.** The Task 1.1 transform lives inside the `Dataset` (`data_loader.py`) and is applied *only* to real images, once per item, before they enter the training loop. DiffAugment (Task 1.5) is applied *inside* the training loop via `prepare_images`, to *both* real images and the generator’s fake images, before they reach the discriminator.
- **Differentiability.** `RandomCrop` and `RandomHorizontalFlip` are non-differentiable PIL operations – this is harmless, because they only ever touch real images that the loss does not back-propagate through. DiffAugment’s transformations (random brightness, colour jitter, translation, cutout) are implemented with differentiable PyTorch ops, so when the generator’s loss $(D(\text{aug}(G(z))) - 1)^2$ is back-propagated, gradients flow through the augmentation back into G . This is essential: if augmentations were applied only to D but were not differentiable, G would be optimised against a discriminator trained on a different distribution from the one G actually sees evaluating itself, and training would drift.
- **Effect on stability and output.** In our experiment the Task 1.1 augmentation alone was insufficient: the baseline run saw D /total collapse to ~ 0.04 and produced samples with strong chromatic artefacts. Adding DiffAugment kept D /total stationary at ~ 0.15 , brought G /loss down from ~ 1.0 to ~ 0.65 , and produced visibly cleaner samples with less fringing and more diversity. The two schemes are complementary rather than competing: standard augmentation grows the effective dataset cheaply for free at load time, while DiffAugment regularises the adversarial game itself by ensuring D never sees the same batch twice.

Part 2 – CycleGAN

Task 2.6 – CycleGAN Results

W&B Report: <https://wandb.ai/nadavabayov-tel-aviv-university/assignment1-cyclegan/reports/Part-2-CycleGAN--VmlldzoxNzEyMjk5OQ>

Sanity Check: 1,000-Iteration Runs

The 1,000-iteration runs behave as expected. The generated images already capture some features of the target domain, confirming the model is learning. Notably, the generator loss G_{yx} is approximately $3\times$ lower in the `cycle_loss=False` run compared to `cycle_loss=True`, which makes sense since the loss is less complex. However, G_{xy} does not show the same notable difference. The reason for this asymmetry between the two directions is unclear.

Qualitative Improvement at 10,000 Iterations

At 10,000 iterations, the generated images show clearer facial features, sharper eyes, and less blurriness overall. The Russian Blue cat generator underperforms compared to the Grumpy Cat generator, which is also reflected in the loss curves where the $Y \rightarrow X$ generator loss remains higher than $X \rightarrow Y$.

Effect of Cycle Consistency Loss

Yes, cycle consistency improves the outputs. Images generated *with* cycle consistency better preserve the original image’s spatial structure — orientation, face shape, and scale. This is expected, since the cycle consistency loss minimizes the ℓ_1 distance between y and $G_{X \rightarrow Y}(G_{Y \rightarrow X}(y))$, explicitly penalizing structural deviations from the input.



Figure 1: Generated images at 1,000 iterations **without** cycle consistency loss.



Figure 2: Generated images at 1,000 iterations **with** cycle consistency loss.

Specific Example from the 1,000-Iteration Grids

Comparing the two 1,000-iteration grids, we can see that across most images in the `cycle_loss=True` grid, the generated faces preserve the orientation, shape, and scale of the original image. In contrast, the `cycle_loss=False` grid generally shows less structural consistency with the source. A clear specific example is the bottom-left image — in the `cycle_loss=True` grid the face orientation and shape closely match the source, while in the `cycle_loss=False` grid the face shape and orientation are noticeably different.

Task 2.7 – Method Improvement

W&B Report: <https://wandb.ai/nadavabayov-tel-aviv-university/assignment1-cyclegan/reports/2-7-Bonus-Part---VmlldzoxNzExMzAxMw>

What We Changed and Why

Looking at the generated images from Task 2.6, they appear quite blurry. This is expected since the L1 cycle consistency loss encourages preserving low frequencies but does not transfer high frequencies well (edges), resulting in blurry outputs. To address this, we added a cycle edge loss:

$$\mathcal{L}_{edge} = \|S(G_{X \rightarrow Y}(x)) - S(x)\|_1 + \|S(G_{Y \rightarrow X}(y)) - S(y)\|_1 \quad (2)$$

where $S(\cdot)$ denotes the Sobel edge detection operator applied in both x and y directions. This encourages the model to preserve high-frequency details, resulting in sharper outputs.

Additionally, we added an identity loss, motivated by the literature which suggests it improves training stability and encourages the generators to be more conservative. Specifically, if an image already belongs to domain Y , then $G_{X \rightarrow Y}$ should not change it significantly, and vice versa:

$$\mathcal{L}_{identity} = \|G_{X \rightarrow Y}(y) - y\|_1 + \|G_{Y \rightarrow X}(x) - x\|_1 \quad (3)$$

Loss Curves

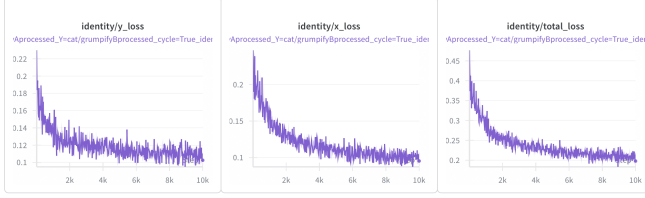


Figure 3: Identity loss curves (y , x , and total) over 10,000 iterations.



Figure 4: Edge loss curves (yxy and xyx) over 10,000 iterations.

Generated Image Samples



Figure 5: Generated images: $X \rightarrow Y$ (Russian Blue $\rightarrow$ Grumpy Cat).



Figure 6: Generated images: $Y \rightarrow X$ (Grumpy Cat $\rightarrow$ Russian Blue).

Discussion of Qualitative Results

The identity loss curves show a smooth and stable descent over 10,000 iterations, and the edge loss plateaus at a stable value, both indicating healthy training. In general, the generated Grumpy Cat images look significantly better compared to the Task 2.6 results — sharper, with more defined eye details. However, the Russian Blue cat generator performs worse compared to Task 2.6, which is also reflected in the loss curves where G_{yx} loss increases from around 5 to 6.

This suggests that adding additional loss terms can sometimes improve one translation direction while hurting the other, as the generators compete over shared capacity.